



Event-Driven Architecture Essentials

Understanding EDA for Modern Systems

2.

Agenda



1. Foundations of Event-Driven Architecture
2. Tools and Techniques for Event-Driven Systems
3. Challenges and Best Practices

3.



Foundations of Event-Driven Architecture

4.



What Is Event-Driven Architecture?



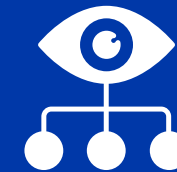
Events Drive Service Communication

Services emit and respond to events, not direct calls.



Loose Coupling Between Components

Services operate independently, reducing dependencies.



Asynchronous Messaging for Scalability

Event flow enables systems to scale and handle spikes.

5.



Why Event-Driven Systems Exist

Breaks Free from Tight Coupling

Services interact via events, not direct calls

Boosts Scalability and Resilience

Asynchronous messaging handles high loads and failures

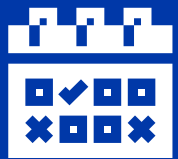
Enables Independent Service Evolution

Teams can update or deploy services without coordination

6.

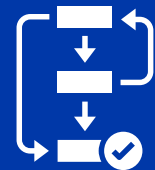


What Is an Event?



Captures a Past Occurrence

Events record facts that have already taken place.



Immutable and Time-Stamped

Once created, events cannot be changed and include a timestamp.



Not Instructions, Just Records

Events document actions, not commands or requests.

7.



How Events Flow Through Systems

Producers Send Events to Broker

Services emit events without targeting specific consumers.

Consumers Subscribe and React

Each consumer independently processes relevant events.

No Direct Producer-Consumer Awareness

Producers remain decoupled from consumer logic and existence.

8.



Types of Events and Commands

Domain Events: Capture Business Changes

Reflect meaningful actions within a specific domain

Integration Events: Enable Service Communication

Facilitate data sharing across service boundaries

Commands: Express Intent, Not Facts

Direct a service to perform an action, unlike events

9.



Tools and Techniques for Event-Driven Systems

10.



Choosing a Message Broker



RabbitMQ: Flexible Queuing & Routing

Ideal for task distribution and complex message flows.



Kafka: High-Throughput Event Streaming

Supports massive data streams and event replay.



Redis Streams: Lightweight Eventing

In-memory, simple setup for internal or small-scale needs.



EVENT-DRIVEN PROGRAMMING WORKFLOW



**BUILDING RESPONSIVE AND DECOUPLED SYSTEMS
BASED ON ASYNCHRONOUS COMMUNICATION**

Pub/Sub and Event Storming

Publish/Subscribe Enables Broad Reaction

Multiple consumers can independently process the same event

Decouples Producers and Consumers

Producers remain unaware of who receives or reacts to events

Event Storming for Domain Discovery

Collaborative workshops reveal key business events

Aligns Technical and Business Teams

Shared understanding of domain events before coding

12.



Challenges and Best Practices

13.

Common Pitfalls in EDA

Event Overload Risks

Too many events can strain infrastructure and slow processing.

Limited Observability

Lack of monitoring makes troubleshooting and tracing difficult.

Schema Inconsistencies

Undisciplined event contracts lead to integration issues.

→ CHALLENGES AND BEST PRACTICES



14.



Key Takeaways on EDA



Events Decouple Services

Reduces dependencies and enables independent scaling.



Message Brokers Power EDA

Central to reliable, scalable event delivery.



Intentional Design is Essential

Discipline prevents chaos and ensures maintainability.